

7-dars: loading data via Pipeline
(Amaliy)

3-project uchun tayyor shablon:

1. Modullar
2. Git Bash (folder + file)
3. Git + Local + Vs Code + Github
4. Standalone + Iehki Git Bash
5. Virtual Env
6. Data Extraction, Merging (3-project u-n
zta yola zta dataset)
7. Logging folder ochish.
8. Katta datasetlarni yuklash
9. Umumiy 2ta Project

Loading Data

Oddiy usul:

- + Oddiy usul $\Rightarrow$ dogsiz
- + Va logli

Class: $\rightarrow$ (Notebooks)

- + src + ipynb (oson, data analysis,
pipeline ucheen ideal emas)
- + src + scripts (industry level)
PY + PY

+ src + scripts + notebooks

Himality

```
import pandas
import os
```

path = r"copy path" # Sababi nested bo'lganligi u-n.

league_name = ['Aserie', 'Bundesliga', 'Laliga', 'Premier League']

Hammasini papkalarini olib oldik. Bu yerda o'zgina mantiqiy va problem solving skills k-k.

seasons = ['1415', '1516', '1617', '1718', '1819', '1920', '2021', '2022'] # bu yerda papkalar ichidagi o'zlash seasonlarni o'zlash joylarini oldik

df_list = []

for league in league_name:

league_folder = os.path.join(path, league)

print(f"League folder not found: {league_folder}")

continue

if not os.path.exists(league_folder)

for season in seasons:

Special case: league 1 1415 has extra underscore

if league == "league 1" and season == "1415":

filename == f"meta-data-league 1 - {season}.csv"

else:

Normal naming for other leagues/seasons

if league == "Arsenal":

filename == f"metadata-series {league} - {season}.csv"

elif league == "Bundesliga":

filename == f"metadata-Bundesliga - {season}.csv"

o

o

o # toki oxirgi paphadawi metadata-largacha elif b/u kiribib chizamiz

else:

continue

file-path = os.path.join(league-folder, filename)

load file only if it exists


```
if os.path.exists(file_path):
```

```
    df = pd.read_csv(file_path)
```

```
    df["league"] = league
```

```
    df["season"] = season
```

```
    df_list.append(df)
```

```
else:
```

```
    # Skip missing files silently or print a message
```

```
    print(f"File not found: {file_path}, skipping.")
```

```
# dogsiz holata info berib ketish
```

```
if df_list:
```

```
    full_df = pd.concat(df_list, index_ignore=True)
```

```
    print("Data loaded successfully.  
Shape: ", full_df.shape)
```

```
else:
```

```
    print("No data loaded")
```

```
# dogli xolatda
```



```
import logging
```

```
# log file path
```

```
log_file = r"copy path of log / file"
```

```
# Ensure the log folder exists
```

```
os.makedirs(os.path.dirname(log_file),  
            exist_ok=True)
```

```
# Setup logging
```

```
logging.basicConfig( # basicConfig bir marta  
                    # set-up gili nadi har doim  
                    # emas
```

```
    filename = log_file,
```

```
    filemode = "a" # append
```

```
    level = logging.INFO,
```

```
    format = "%(asctime)s - %(levelname)s -  
              %(message)s"
```

```
)
```

```
# Test log messages
```

```
logging.info("☑ logging is working")
```

```
logging.warning("⚠ This is a warning test")
```

```
logging.error("✖ This is an error test")
```

```
print(f"logging test completed. Check the  
      file: {log_file}")
```


No.
roglidagi fagat tepadagi ligolar
misolida

```
import os  
import pandas as pd
```

```
# Base path
```

```
path = + "copy path too"  
league_name = ["ASeril", "BundesLigue",  
"dahiga", "league1", "Premier league"]  
seasons = ["1415", "1516", "1617", "1718",  
"1819", "1920", "2021", "2122"]
```

```
df_list = []
```

```
for league in league_name:  
    league_folder = os.path.join(path, league)  
    if not os.path.exists(league_folder):  
        logging.warning(f"league folder not  
found: {league_folder}")  
        continue
```

```
for season in seasons:  
    # Special case: league1 1415 has extra underscore  
    if league == "league1" and season ==  
        "1415":  
        filename = f"metadata-league1-  
{season}.csv"  
    else:
```



```
if league == "Aserie" : series-a-  
    filename = f"metadata-league-{season}.  
                .csv"
```

```
elif league == "Bundesliga" :  
    filename = f"metadata-bundesliga-  
                {season}.csv"
```

o

```
# to tugaguncha yozamiz
```

```
else:
```

```
    continue
```

```
file_path = os.path.join(league_folder,  
                           filename)
```

```
# load file only if it exists
```

```
if os.path.exists(file_path):
```

```
    try:
```

```
        df = pd.read_csv pd.read_csv(  
            file_path)
```

```
        df["league"] = league
```

```
        df["season"] = season
```

```
        df_list.append(df)
```

```
        logging.info(f"loaded file: {file-  
                        path}")
```

```
    except Exception as e:
```

```
        logging.error(f"Error loading file  
                        {file_path}: {e}")
```

```
else:
```

```
    logging.warning(f"file not found: {file_path}, skipping")
```

Concatenate all loaded DataFrames

if df_list:

try:

full_df = pd.concat(df_list,
ignore_index=True)

logging.info(f"Data xatolaringiz yuk-
landi. Shape: {full_df.shape}")

except Exception as e:

logging.error(f"Error concatenating
dataframes: {e}")

else:

logging.error("No data loaded! Check
your file paths and filenames.")

Loading data using class

+ Toza va tartibli koding uchun kerak bo'ladigan jarayon

+ Kerakli papkalar:

- src + notebooks

- src + scripts

Kerakli ishlar:

- Pip install `pickleshare` # class = nameni import qilib dasturlashda xatolik bermashi uchun.

- src papkasini ichiga `--init--.py`

↳ folder → package;

↳ Subfolder allows;

↳ control

fmalily: data_loader.py

import pandas as pd

import os

import logging

class DataLoader:

def __init__(self, path):
self.path = path

With log bir xil proses foydalanishda birlashtirish qismi bor

~~class~~

def load_datasets(self):

Concatenate all loaded DataFrames

return full_df

return pd.DataFrame()

Shuni endi notebooks'da chagirishni
koramiz

```
import sys
import os
```

```
# class uchun path
sys.path.append(r"copy path of src")
```

```
# class nameni file dan import qiling
from data_loader import DataLoader
```

```
# dataset path
```

```
loader = DataLoader(r"copy of dataset path")
↳ # Siz loader = DataLoader("manzil") deb yozgan in RAW folder")
```

```
# load all datasets pytingirda, aslida path argumentini  
chagirib bilgansiz.
```

```
full_df = loader.load_datasets()
```

```
# With dog
```

```
import sys
import os
import logging
```

```
# 1. Configure logging
```

```
logging.basicConfig(
```

```
    filename = r"copy path of log file where it's needed",
```

```
    filemode = "w"
```

```
    level = logging.INFO,
```

```
    format = "%(asctime)s - %(levelname)s - %(message)s"
```

```
)
```


2. add src to path

```
sys.path.append(r"copy path of src folder")
```

3. Import Delta loader

```
from data_loader import DataLoader
```

4. Initialize with dataset path

```
loader = DataLoader(r"copy path of datasets  
folder of psu_data")
```

shu yerdan path berildi ammo bilib olginki. ↪
biz = xom datami qayerda tusa uni pathini
o'sha yerga berish k-k!

5. load datasets

```
full_df = loader.load_datasets()
```

```
print(full_df.shape)
```

src + scripts => Billa ishlatilishini
ko'ramiz

Bunda huddi notebooks dagini scriptsda
.py fayl b/n qilar ekanmiz va uni sonab
ko'rish uchun

→ terminaldan => ~~src~~ cd src, undan keyin
lsa python data-load.py desak terminal
da chiqadi.

bu holatda ko'pincha industrial xolatda ishlatiladigan ishdir. src + scripts ishlatish hozirgi loyihadagi holat bo'ladi.

Pipeline

- Mustaqil bo'lgan bir nechta qism/ni ma'lum bir vazifa uchun ketma ket birlashtirish jarayoni
- Pipelineda birinchi xolat = outputi o'zidan keyingi jarayon uchun input bo'ladi.
- Pipelineda har bir qism (modeli) ma'lum bir ishni bajaradi

Key features:

- Modularity => Papkalardan tashkil topgan bo'ladi
- Reusability => Har birini alohida ishlatish bo'ladi
- Clear Separations of concerns => Alohida-alohida qilib toza ishloshni ta'minlaydi.

Qanday foydasi bor:

- Tushunarli va toza workflow, oson xatolarni topish. Qayta ishlatish bo'ladigan kodlar blogi va toza d'rolari alohida project qilib bu ish bo'lish imkoniyati bo'zligi.

Amaliy : Pipeline

1. lowercase (kichkina harflarda bajartsin)

```
def to_lower(text):  
    return text.lower()
```

2. Orasidagi empty spaclarni toralashtir

```
def remove_space(text):  
    return text.replace(" ", "")
```

3. Teskaridan yozib bersin

```
def reverse(text):  
    return text[::-1]
```

pipeline da

```
pipeline = [to_lower, remove_space, reverse]  
data = "Hello AI"
```

```
for step in pipeline:  
    data = step(data)  
print(data)
```